UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026).
   Answers are unofficial reference answers, not official UTM marking schemes.

2. Marking points show the expected answer and mark allocation; check them against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Approved real questions only; no generators, placeholders or parametric variants.
   Questions and answers are for local study use only.

5. Structure derived from the supplied CSC429 papers; every question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 11 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION 1 (5 marks)

Source: CSC429-2015-12, page [4]

Unofficial model answer:

a) Three types of system interconnection and their functions: (1) Bus interconnection: A shared set of parallel electrical conductors (address, data, and control lines) that connects the CPU, memory, and I/O modules. Its function is to provide a common communication path for transferring data, addresses, and control signals among system components; only one master can use the bus at a time, so arbitration is required. (2) Point-to-point interconnection: Dedicated links (e.g., serial lanes or switched fabric) between two components, such as CPU-to-memory or CPU-to-I/O controller. Its function is to provide high-bandwidth, low-latency, private communication without contention from other devices, often used in modern systems (e.g., PCI Express, QuickPath Interconnect). (3) Crossbar switch / switched interconnection: A matrix of switches that can simultaneously connect multiple pairs of ports (e.g., memory-to-CPU and I/O-to-memory). Its function is to allow multiple concurrent transfers, increasing overall system throughput and reducing contention compared with a shared bus. (Other acceptable: multistage interconnection network, ring, mesh; describe function accordingly.)

b) Physical appearance difference: System interconnection is the physical wiring/backplane (or set of links and switches) that spans the whole computer system, connecting the CPU, main memory, and I/O modules; it is typically a set of parallel traces on a motherboard, cables, or a switch fabric, and is visible as a shared bus or network of links. CPU interconnection, in contrast, is the internal wiring within the CPU chip (or between CPU cores) that connects the control unit, ALU, registers, and cache; it is microscopic, on-die, and not visible as external cabling. Thus, system interconnection is macroscopic and external to the CPU, while CPU interconnection is microscopic and internal to the processor.

Marking points:

- *1 mark: Correctly names and describes one type of system interconnection (e.g., bus) and its function.*
- *1 mark: Correctly names and describes a second type (e.g., point-to-point) and its function.*
- *1 mark: Correctly names and describes a third type (e.g., crossbar switch) and its function.*
- *1 mark: States that system interconnection is external/macroscopic, connecting CPU, memory, and I/O (e.g., motherboard traces, cables, backplane).*
- *1 mark: States that CPU interconnection is internal/microscopic, on-chip wiring between CPU components (control unit, ALU, registers, cache) or between cores.*

Model confidence: medium.

Model note: The question text does not specify a particular textbook taxonomy; the answer uses common computer-organization categories (bus, point-to-point, crossbar/switched). If the course used different terminology (e.g., 'processor-memory bus', 'I/O bus', 'interconnect network'), those should be accepted. No figure or table was supplied, so the physical appearance description is based on general knowledge.

QUESTION 2 (5 marks)

Source: CSC429-2014-06, page [2]

Unofficial model answer:

a) In the **third** Generation, the need for air conditioning is not compulsory. (Third-generation computers used integrated circuits, which generated less heat than vacuum tubes or discrete transistors, so air conditioning was not mandatory.)

b) The acronym EDSAC stands for **Electronic Delay Storage Automatic Calculator**. (EDSAC was a first-generation stored-program computer.)

© Hak Cipta Universiti Teknologi MARA

c) Ultra Large Scale Integration (ULSI) is one of the features in the **fourth** Generation. (Fourth-generation computers are characterised by microprocessors and VLSI/ULSI circuitry.)

d) **John von Neumann** is known as the father of modern computer. (He proposed the stored-program concept.)

e) Gottfried von Leibniz is the inventor of the **Stepped Reckoner** (also called the Leibniz wheel/stepped-drum) mechanical machine.

Marking points:

• *1 mark: a) third*

• *1 mark: b) Electronic Delay Storage Automatic Calculator*

• *1 mark: c) fourth*

• *1 mark: d) John von Neumann*

• *1 mark: e) Stepped Reckoner (Leibniz wheel/stepped-drum calculator)*

Model confidence: high.

Model note: The question text is partially garbled (e.g., 'EDSAC stands for ___ Generation' and 'Ultra Large Scale Integrator' likely means Ultra Large Scale Integration). Answers are based on standard CSC429 computer-generation facts. No official marking scheme is implied.

QUESTION 3 (5 marks)

Source: CSC429-2025-02, page [3]

Unofficial model answer:

a) Sign and magnitude: Convert 40h to decimal: 4×16 = 64. So 408 − 40h = 408 − 64 = 344. In sign and magnitude, 344 in binary is 101011000 (9 bits). To represent as a signed number, we need a sign bit: 0 for positive. Using 10 bits: sign bit 0, magnitude 101011000 → 0 101011000. Thus result = 0101011000_2 (sign-magnitude). Workflow: 408_{10} = 110011000_2 (9 bits), 40h = 64_{10} = 1000000_2 (7 bits). Align to 10 bits: 408 = 0110011000 (sign 0, magnitude 110011000), 64 = 0001000000 (sign 0, magnitude 0001000000). Subtract magnitudes: 110011000 − 0001000000 = 101011000 (344). Since 408 > 64, result positive, sign bit 0. Final: 0 101011000 = 0101011000_2.

b) 1's and 2's complement: Compute 39h − 23a. Convert to decimal: 39h = 3×16+9 = 57; 23a is not valid hex (contains 'a'? Actually 'a' is valid hex digit 10, but 23a has three digits: 2,3,a → 2×256 + 3×16 + 10 = 512+48+10=570). So 39h = 57_{10}, $23a_{16}$ = 570_{10}. Result = 57 − 570 = −513. But typical 8-bit? 23a exceeds 8 bits. Use 16-bit representation. 39h = 0x0039 = 0000 0000 0011 1001_2. 23a = 0x023A = 0000 0010 0011 1010_2. 1's complement of 23a: 1111 1101 1100 0101. 2's complement: 1111 1101 1100 0110. Add to 39h: 0000 0000 0011 1001 + 1111 1101 1100 0110 = 1111 1110 0000 0000? Let's compute: 0x0039 + 0xFD C6 = 0xFE00? Actually 0x0039 + 0xFDC6 = 0xFDFF? Wait: 0x0039 + 0xFDC6 = 0xFDFF? 0x39+0xC6=0xFF, 0x00+0xFD=0xFD → 0xFDFF. That is not −513. Let's recalc: 23a hex = 0x023A. 1's complement: 0xFD C5 (since 0x023A → 1111 1101 1100 0101 = 0xFDC5). 2's complement: 0xFDC6. Add 0x0039: 0x0039 + 0xFDC6 = 0xFDFF. 0xFDFF in 2's complement = −(0x0201) = −513? 0x0201 = 513, so 0xFDFF = −513? Check: 0xFFFF = −1, 0xFDFF = −(0x0201) = −513. Yes correct. So result = 0xFDFF (2's complement) = -513_{10}. For 1's complement method: add 39h + 1's complement of 23a = 0x0039 + 0xFDC5 = 0xFDFE. Since there is no carry out, result is negative and in 1's complement form: 0xFDFE = −(0x0201) = −513? 1's complement of 0xFDFE is 0x0201 = 513, so value = −513. So final answer: 1's complement: 0xFDFE; 2's complement: 0xFDFF. Workflow: represent both in 16-bit binary, take complement of subtrahend, add, interpret sign.

Marking points:

• *1 mark: Correctly convert 40h to decimal 64 and compute 408-64=344.*

• *1 mark: Show sign-magnitude representation of 344 as 0 101011000 (or equivalent with sign bit).*

• *1 mark: Correctly interpret 39h as 57 and 23a as 570 (or note 23a is 3 hex digits).*

• 1 mark: Show 1's complement of 23a (0xFDC5) and addition to get 0xFDFE, interpret as -513.

• 1 mark: Show 2's complement of 23a (0xFDC6) and addition to get 0xFDFF, interpret as -513.

Model confidence: medium.

Model note: Assumption: 23a is a 3-digit hexadecimal number (0x23A). If the question intended 23a as a typo for 23h, answer would differ. Also assumed 16-bit word length for complement method; if 8-bit, 23a cannot be represented. Sign-magnitude part assumed 10-bit representation to accommodate 344 (9 bits) plus sign bit.

QUESTION 4 (5 marks)

Source: CSC429-2016-12, page [3]

Unofficial model answer:

a) One device that can be both an input and output device is a touchscreen (or touch-sensitive display). [1 mark]

b) Input scenario: When a user taps an on-screen button or types on a virtual keyboard, the touchscreen senses the physical touch and sends the corresponding coordinate and action data to the computer, acting as an input device. Output scenario: When the computer displays text, images, or a graphical user interface on the same screen, the touchscreen presents visual information to the user, acting as an output device. [2 marks]

c) A basic Von Neumann architecture sketch should show the five main units: Input unit, Output unit, Memory unit, Arithmetic Logic Unit (ALU), and Control Unit, with the CPU comprising the ALU and Control Unit. The units are connected by a system bus (address bus, data bus, control bus). Arrows should indicate data flow: input to memory/CPU, CPU to memory, CPU to output, and control signals from the control unit. The stored-program concept is implied by memory holding both instructions and data. [2 marks]

Marking points:

• 1 mark: Correctly states one device that is both input and output (e.g., touchscreen, touch-sensitive display, modem, network interface card).

• 1 mark: Describes a valid input scenario for the device (e.g., user touches screen to provide input).

• 1 mark: Describes a valid output scenario for the device (e.g., screen displays information to user).

• 1 mark: Sketch includes the five classical Von Neumann units: input, output, memory, ALU, and control unit (CPU may be shown as ALU + control unit).

• 1 mark: Sketch includes appropriate connections (e.g., bus or arrows) and labels showing data/control flow between units.

Model confidence: high.

Model note: The question asks for a sketch; the answer describes the required components and connections. No figure is provided in the extracted text, so the sketch is described in words. The marking scheme is an unofficial model answer and not an official UTM marking scheme.

QUESTION 5 (10 marks)

Source: CSC429-2023-07, page [3]

Unofficial model answer:

a) The main function of cache memory is to reduce the average time to access main memory by holding frequently accessed instructions and data in a small, fast memory close to the CPU. It exploits locality of reference (temporal and spatial) so that most CPU requests are satisfied from cache rather than slower main memory, thereby bridging the speed gap between the processor and main memory and improving overall system performance.

b) Data in cache and main memory are synchronized by using a write policy. In write-through, every write to cache is also written immediately to main memory, keeping them consistent at the cost of more memory traffic. In write-back, writes update only the cache block and set a dirty bit; the block is written back to main memory

© Hak Cipta Universiti Teknologi MARA

only when it is replaced, reducing traffic but requiring a dirty-bit mechanism. Cache coherence protocols (e.g., snooping or directory-based) are also used in multiprocessor systems to keep multiple caches and main memory consistent.

c) SRAM vs DRAM: (i) Type of storage: SRAM stores each bit in a bistable flip-flop (static), while DRAM stores each bit as charge on a capacitor (dynamic). (ii) Availability of charge leak: SRAM has no charge leakage because it uses active transistors holding state, whereas DRAM suffers charge leakage from capacitors and must be refreshed periodically. (iii) Price: SRAM is more expensive per bit (larger cell area, more transistors), while DRAM is cheaper per bit (smaller cell, one transistor plus capacitor). (iv) Example of location of memory usage: SRAM is used in CPU cache and registers; DRAM is used for main memory (RAM modules).

Marking points:

• *1 mark: cache reduces average memory access time / bridges CPU-memory speed gap*
• *1 mark: exploits locality of reference (temporal/spatial)*
• *1 mark: write-through policy keeps cache and main memory consistent*
• *1 mark: write-back policy with dirty bit writes back on replacement*
• *1 mark: cache coherence protocols for multiprocessor synchronization*
• *1 mark: SRAM type of storage = bistable flip-flop / static*
• *1 mark: DRAM type of storage = capacitor charge / dynamic*
• *1 mark: SRAM no charge leakage vs DRAM charge leakage and refresh*
• *1 mark: SRAM more expensive per bit vs DRAM cheaper per bit*
• *1 mark: SRAM used in cache/registers vs DRAM used in main memory*

Model confidence: high.

Model note: Unofficial model answer; not an official UTM marking scheme. No figure/table was supplied beyond the question text, so the answer is based on standard computer-organization concepts.

QUESTION 6 (10 marks)

Source: CSC429-2024-07, page [4]

Unofficial model answer:

a) The four classifications of parallel processors (Flynn's taxonomy) are: (1) SISD — Single Instruction, Single Data: a conventional uniprocessor such as a standard PC executing one instruction stream on one data stream. (2) SIMD — Single Instruction, Multiple Data: one control unit drives many processing elements, e.g., array processors, vector processors, and GPU/SIMD extensions (MMX/SSE/AVX). (3) MISD — Multiple Instruction, Single Data: multiple instruction streams operate on the same data stream; it is rarely used in practice, with examples such as systolic arrays or fault-tolerant systems with replicated processing. (4) MIMD — Multiple Instruction, Multiple Data: multiple processors execute independent instruction streams on independent data, e.g., multicore processors, SMP servers, and clusters/distributed systems. b) One advantage of cluster computer architecture is high availability/fault tolerance: because a cluster consists of multiple independent computers connected by a network, if one node fails the remaining nodes can continue providing service (failover), so the system as a whole is more reliable than a single large machine. A second acceptable advantage is scalability/cost-effectiveness: commodity nodes can be added incrementally to increase performance at lower cost than a single high-end supercomputer.

Marking points:

• *1 mark: SISD correctly named with a valid example (e.g., uniprocessor/PC).*
• *1 mark: SIMD correctly named with a valid example (e.g., vector/array processor, GPU, SIMD extensions).*
• *1 mark: MISD correctly named with a valid example (e.g., systolic array, replicated fault-tolerant system).*
• *1 mark: MIMD correctly named with a valid example (e.g., multicore, SMP, cluster).*
• *2 marks: all four classifications correctly named (0.5 each) — total for part (a) = 6 marks.*
• *2 marks: one clear advantage of cluster architecture stated (e.g., high availability/fault tolerance or scalability/cost-*

© Hak Cipta Universiti Teknologi MARA

effectiveness).

• 2 marks: advantage explained with justification (e.g., node failure does not stop service; commodity nodes can be added incrementally).

• Total: 10 marks.

Model confidence: high.

Model note: Assumes the question refers to Flynn's taxonomy, the standard classification taught for parallel processors. If the course used a different taxonomy (e.g., shared-memory vs distributed-memory), the named categories would differ; the marking scheme should be adjusted accordingly. No figure or table was supplied.

QUESTION 7 (10 marks)

Source: CSC429-2019-12, page [4]

Unofficial model answer:

Registers are small, high-speed storage locations inside the CPU used to hold data, instructions, and addresses during program execution. Their specific functions are: (1) Program Counter (PC): holds the address of the next instruction to be fetched; it is automatically incremented after each fetch so the CPU knows where to continue. (2) Instruction Register (IR): holds the instruction currently being decoded and executed; its contents are fed to the control unit. (3) Memory Address Register (MAR): holds the address of the memory location to be accessed (read or written) by the memory unit. (4) Memory Data Register (MDR) / Memory Buffer Register (MBR): holds the data read from or to be written to memory; it acts as a buffer between the CPU and main memory. (5) Accumulator (AC): a general-purpose register that holds intermediate arithmetic and logic results and often one operand of an ALU operation. (6) General-Purpose Registers (GPRs): a set of registers (e.g., R0–R15) used by the programmer for holding operands, intermediate values, and addresses; they reduce memory accesses. (7) Status/Flag Register (PSW): holds condition flags (zero, carry, sign, overflow) and control bits that record the outcome of ALU operations and CPU state. (8) Stack Pointer (SP): holds the address of the top of the stack in memory, used for subroutine calls, returns, and push/pop operations. (9) Index Register: holds an offset used in indexed addressing to compute effective addresses. (10) Temporary registers (e.g., in the datapath) hold operands and results during instruction execution inside the CPU. Each register is typically a fixed-width flip-flop array directly accessible by the control unit, giving fast access compared with cache or main memory.

Marking points:

• 1 mark: Correct definition of a register as small, fast CPU storage.

• 1 mark: Program Counter (PC) – holds address of next instruction.

• 1 mark: Instruction Register (IR) – holds current instruction being decoded/executed.

• 1 mark: Memory Address Register (MAR) – holds memory address for access.

• 1 mark: Memory Data Register (MDR/MBR) – buffers data to/from memory.

• 1 mark: Accumulator (AC) – holds intermediate ALU results/operand.

• 1 mark: General-purpose registers – programmer-visible operands/intermediate values.

• 1 mark: Status/Flag register (PSW) – condition flags and CPU state.

• 1 mark: Stack Pointer (SP) – points to top of stack for calls/returns.

• 1 mark: Any other valid register (index, temporary, instruction pointer) with correct function.

Model confidence: high.

Model note: No figure or table was supplied; answer is based on standard computer-organization register terminology. Marking assumes one mark per correctly described register/function up to 10 marks; if the official scheme groups registers differently, adjust allocation accordingly. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 8 (10 marks)

Source: CSC429-2019-12, page [3]

Unofficial model answer:

The question asks for the subtraction FIFAie − 664i6 (i.e., 0xFIFA − 0x6646) using two techniques, showing the complete workflow. Both operands are 16-bit hexadecimal values: A = 0xFIFA, B = 0x6646.

(i) Sign and magnitude: In sign-and-magnitude the MSB is the sign bit (1 = negative, 0 = positive) and the remaining 15 bits are the magnitude. A = 0xFIFA = 1111 1111 1111 1010_2 has sign bit 1 (negative) and magnitude 111 1111 1111 1010_2 = 0x7FFA = 32762_{10}, so A = −32762. B = 0x6646 = 0110 0110 0100 0110_2 has sign bit 0 (positive) and magnitude 110 0110 0100 0110_2 = 0x6646 = 26182_{10}, so B = +26182. Workflow: (1) decode sign and magnitude of each operand; (2) since signs differ, subtract the smaller magnitude from the larger: 32762 − 26182 = 6580; (3) the result takes the sign of the operand with the larger magnitude, i.e. negative; (4) encode 6580_{10} = 0x19B4 = 001 1001 1011 0100_2 in 15 bits and set the sign bit to 1, giving 1001 1001 1011 0100_2 = 0x99B4. Check: −32762 − 26182 = −58944, which is outside the 16-bit sign-and-magnitude range (−32767 to +32767), so the true result overflows; the 16-bit pattern 0x99B4 represents −6580, which is the wrapped/incorrect result. The correct mathematical answer is −58944, and overflow must be flagged.

(ii) Two's complement: Workflow: (1) write both operands in 16-bit binary: A = 1111 1111 1111 1010, B = 0110 0110 0100 0110; (2) subtraction A − B is performed as A + (−B), so form the two's complement of B: invert B → 1001 1001 1011 1001, add 1 → 1001 1001 1011 1010 (0x99BA); (3) add A + (−B): 1111 1111 1111 1010 + 1001 1001 1011 1010 = 1 1001 1001 1011 0100; discard the carry-out beyond 16 bits, leaving 1001 1001 1011 0100 = 0x99B4; (4) interpret: MSB = 1 so the result is negative; take the two's complement to find the magnitude: invert 1001 1001 1011 0100 → 0110 0110 0100 1011, add 1 → 0110 0110 0100 1100 = 0x664C = 26188, so the stored result is −26188. Check overflow: the operands have opposite signs after negation (A negative, −B negative), so adding two negatives that yield a positive-sign carry pattern indicates overflow; indeed −32762 − 26182 = −58944 lies outside the 16-bit two's complement range (−32768 to +32767), so the 16-bit result 0x99B4 is invalid and the overflow flag V = 1. The correct result requires more than 16 bits (e.g., 32-bit two's complement gives 0xFFFF19C0 = −58944).

Note: the question text 'FIFAie - 664i6' is a garbled rendering of hexadecimal operands; the answer assumes 0xFIFA − 0x6646 as the intended values.

Marking points:

• 1 mark: correctly identify operands as 16-bit hex 0xFIFA and 0x6646 (or state the assumption about the garbled notation)

• 1 mark (i): decode 0xFIFA in sign-and-magnitude as sign = 1 (negative), magnitude 0x7FFA = 32762

• 1 mark (i): decode 0x6646 as sign = 0 (positive), magnitude 26182

• 1 mark (i): subtract magnitudes (32762 − 26182 = 6580) and assign negative sign from the larger-magnitude operand

• 1 mark (i): encode result as 0x99B4 and note that the true answer −58944 overflows the 16-bit sign-and-magnitude range

• 1 mark (ii): write both operands in 16-bit binary and form two's complement of 0x6646 = 0x99BA

• 1 mark (ii): perform binary addition 0xFIFA + 0x99BA, discard carry-out, obtain 0x99B4

• 1 mark (ii): interpret 0x99B4 as negative by taking two's complement to get magnitude 0x664C = 26188, i.e. −26188

• 1 mark (ii): state overflow condition (adding two negatives gives positive-sign result / true result −58944 outside −32768..+32767) and set V = 1

• 1 mark: overall correct workflow shown (sign handling, complement step, addition, interpretation) and consistent final statement that the 16-bit result is invalid due to overflow

Model confidence: medium.

Model note: The question text 'FIFAie - 664i6' is garbled OCR/extraction; the answer assumes the intended operands are hexadecimal 0xFIFA and 0x6646. If the intended values differ, the arithmetic steps remain valid but the numeric results change. No figure or table was supplied. This is an unofficial model answer, not an official UTM marking scheme.

© Hak Cipta Universiti Teknologi MARA

QUESTION 9 (10 marks)

Source: CSC429-2024-07, page [3]

Unofficial model answer:

a) Two types of bus timing: (1) Synchronous bus timing — all bus transactions are governed by a common clock; events occur at fixed clock edges, so the timing of address, data and control signals is predetermined. It is simple and fast for devices with similar speeds, but the bus must run at the speed of the slowest device, and slow devices need wait states. (2) Asynchronous bus timing — no common clock; a handshake protocol (e.g., request/acknowledge or master strobe/slave acknowledge) coordinates the transfer, so each device proceeds at its own speed. It accommodates both fast and slow devices and is more flexible, but the handshake adds control overhead and complexity. (A third type, semi-synchronous, is a hybrid using a clock plus handshake, but only two are required.)

b) Three CPU components: (1) Control Unit (CU) — fetches and decodes instructions and generates the control signals that direct the datapath, ALU, registers and memory/I/O operations; it sequences the fetch–decode–execute cycle. (2) Arithmetic Logic Unit (ALU) — performs arithmetic operations (add, subtract, etc.) and logical operations (AND, OR, NOT, XOR, shifts/compares) on operands supplied from registers. (3) Registers — small, fast storage inside the CPU, including the Program Counter (PC), Instruction Register (IR), Memory Address Register (MAR), Memory Data Register (MDR) and general-purpose/accumulator registers; they hold the current instruction, addresses, operands and results. (The internal buses connecting these may also be mentioned.)

Marking points:

- *a) 1 mark: correctly naming synchronous bus timing*
- *a) 1 mark: explaining synchronous timing (common clock, fixed clock edges, wait states / slowest-device limitation)*
- *a) 1 mark: correctly naming asynchronous bus timing*
- *a) 1 mark: explaining asynchronous timing (no common clock, handshake/request-acknowledge, accommodates different device speeds)*
- *a) 1 mark: valid comparison/advantage-disadvantage of the two timing types*
- *b) 1 mark: naming Control Unit*
- *b) 1 mark: describing Control Unit function (instruction fetch/decode, control-signal generation, sequencing)*
- *b) 1 mark: naming ALU*
- *b) 1 mark: describing ALU function (arithmetic and logical operations)*
- *b) 1 mark: naming registers (PC/IR/MAR/MDR/general-purpose) and describing their role*

Model confidence: high.

Model note: Unofficial model answer, not an official UTM marking scheme. No figure/table was supplied; answer is based on standard CSC429 bus-timing and CPU-component material. Mark allocation is indicative: 5 marks for part (a) and 5 marks for part (b), with sub-points as listed.

QUESTION 10 (10 marks)

Source: CSC429-2016-12, page [5]

Unofficial model answer:

a) Three main key factors for memory selection: (1) Cost per bit: Memory technology is chosen to balance cost against capacity; SRAM is fast but expensive per bit, DRAM is cheaper per bit, and magnetic/optical storage is cheapest per bit, so a memory hierarchy uses expensive fast memory for cache and cheaper slower memory for main memory and secondary storage. (2) Access time (latency): The time to read/write a memory location affects overall system performance; faster access time is needed for cache and registers, while slower access time is acceptable for main memory and disk. (3) Capacity: The amount of memory required to hold the working set and programs; larger capacity generally increases cost and may increase access time, so capacity is selected to meet application and OS requirements. Other acceptable factors include volatility, erasability,

© Hak Cipta Universiti Teknologi MARA

and physical size/form factor, but the three above are the main ones.

b) Two examples of internal cabling memory: (i) registers (CPU registers), (ii) cache memory (L1/L2/L3). Two examples of system memory: (i) main memory (RAM, e.g., DRAM), (ii) secondary storage (hard disk drive, solid-state drive, or ROM). Note: 'internal cabling memory' is interpreted as memory internal to the CPU/system that is directly wired/connected, i.e., registers and cache; 'system memory' refers to memory used by the system for programs and data, i.e., main memory and secondary storage.

Marking points:

• 1 mark: Correctly identifies cost per bit as a key factor and briefly explains the trade-off between cost and speed/capacity.

• 1 mark: Correctly identifies access time (latency) as a key factor and explains its impact on performance.

• 1 mark: Correctly identifies capacity as a key factor and explains the need to match application/OS requirements.

• 1 mark: Additional credit for mentioning other valid factors (e.g., volatility, erasability, physical size) up to the 3-factor limit.

• 1 mark: Gives one correct example of internal cabling memory (e.g., registers).

• 1 mark: Gives a second correct example of internal cabling memory (e.g., cache).

• 1 mark: Gives one correct example of system memory (e.g., main memory/RAM).

• 1 mark: Gives a second correct example of system memory (e.g., secondary storage/ROM).

• 1 mark: Clarity and correct use of terminology in part (a).

• 1 mark: Clarity and correct categorization in part (b).

Model confidence: medium.

Model note: The term 'internal cabling memory' is non-standard; it is assumed to mean memory internal to the CPU/system that is directly connected (registers and cache). If the examiner intended a different meaning, part (b) examples may vary. The answer is based on standard computer organization textbooks (e.g., Stallings, Patterson & Hennessy).

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION 11 (20 marks)

Source: CSC429-2025-02, page [6]

Unofficial model answer:

The equation in Figure 2 is interpreted as y = A - (2C * D) / (B + A^2). Registers A, B, C, D are fixed; x, y, z are temporary; final result must be in y. Table 1 provides one-address (LOADp, STORp, ADDp, SUBp, MULp, DIVp), two-address (MOV(p,q), ADD(p,q), SUB(p,q), MUL(p,q), DIV(p,q)), and three-address (ADD(p,q,r), SUB(p,q,r), MUL(p,q,r), DIV(p,q,r)) instructions. In two- and three-address notations, the first operand is the destination. For one-address, the accumulator is implicit; we use x as accumulator and y as final storage.

One-address program: LOAD A; MUL A; STOR z; LOAD B; ADD z; STOR z; LOAD C; MUL C; MUL D; STOR x; LOAD A; SUB x; STOR y; LOAD x; DIV z; STOR y. Explanation: compute A^2 into z, then B+A^2 into z; compute 2C*D into x; compute A - x into y; finally divide by z and store in y. This uses only the listed one-address instructions and temporary registers x, y, z.

Two-address program: MOV(x, A); MUL(x, A); ADD(x, B); MOV(z, C); MUL(z, C); MUL(z, D); MOV(y, A); SUB(y, z); DIV(y, x). Explanation: x = A^2 + B; z = 2C*D; y = A - z; y = y / x. The final result is in y. All instructions are from the two-address column.

Three-address program: MUL(x, A, A); ADD(x, x, B); MUL(z, C, C); MUL(z, z, D); SUB(y, A, z); DIV(y, y, x). Explanation: x = A^2 + B; z = 2C*D; y = A - z; y = y / x. The final result is in y. All instructions are from the three-address column.

Note: The figure was supplied only as an image asset; the equation is inferred from the extracted text 'y = A - 2C * D / (B + A2)'. If the intended grouping differs (e.g., (2C*D)/(B+A^2) versus 2C*(D/(B+A^2))), the instruction sequences would change accordingly. The answer assumes standard operator precedence and left-to-right evaluation for multiplication/division.

Marking points:

*• 1 mark: Correctly interpret the equation as y = A - (2C * D) / (B + A^2) or equivalent, showing understanding of operator precedence.*

• 1 mark: State that one-address instructions use an implicit accumulator and that x, y, z are used as temporary/final storage.

• 2 marks: One-address program correctly computes A^2 and B+A^2 using LOAD, MUL, ADD, STOR (partial credit for minor sequencing errors).

*• 2 marks: One-address program correctly computes 2C*D using LOAD, MUL, MUL, STOR.*

*• 2 marks: One-address program correctly computes A - (2C*D) and divides by (B+A^2), storing final result in y.*

• 1 mark: Two-address program correctly computes x = A^2 + B using MOV, MUL, ADD.

*• 1 mark: Two-address program correctly computes z = 2C*D using MOV, MUL, MUL.*

• 2 marks: Two-address program correctly computes y = A - z and y = y / x, final result in y.

• 1 mark: Three-address program correctly computes x = A^2 + B using MUL, ADD.

*• 1 mark: Three-address program correctly computes z = 2C*D using MUL, MUL.*

• 2 marks: Three-address program correctly computes y = A - z and y = y / x, final result in y.

• 1 mark: All instructions used are from Table 1 and correct assembly notation (operand order, register names) is followed.

• 1 mark: Overall clarity, correct use of temporary registers, and final result stored in y for all three programs.

• 1 mark: Acknowledges any ambiguity in the figure/equation (e.g., grouping) and states assumptions.

Model confidence: medium.

*Model note: The equation was inferred from the extracted text 'y = A - 2C * D / (B + A2)' because the figure was provided only as an image asset without a machine-readable equation. The answer assumes standard precedence: multiplication and division before subtraction, and left-to-right for * and /. If the intended equation is different (e.g., y = A - 2C * (D/(B+A^2))), the programs would*

© Hak Cipta Universiti Teknologi MARA

need adjustment. Table 1 was provided as a native table and is used as-is. This is an unofficial model answer, not an official UTM marking scheme.

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA